Examflow

Deployment Guide

A complete, step-by-step guide to putting your hall ticket and attendance system online — written assuming you have no technical background. Follow it top to bottom in order.

Your questions, answered up front

Where do I deploy this?	Render.com for the website + backend, and Neon.tech for the database — both free.
Do I need Vercel?	No. Vercel can't run this app's backend or database — Render does everything else in one place.
Do I need to buy a domain (DNS)?	No. Render gives you free working web addresses with security (HTTPS) built in.
How do students get emailed?	Through your Gmail/Google Workspace account, using a special "app password" (Part 1).
How much will it cost?	$0/month — since you send emails in weekly bursts rather than continuously, the free tiers fit your usage perfectly (see "A note on cost" below).

A note on cost: you mentioned you'll send your ~1200 emails in a weekly burst rather than continuously — so this guide uses free tiers throughout instead of "always-on" paid ones. The only tradeoff: if nobody's used the site in the last 15 minutes, the next visit takes about 30-60 seconds to "wake up" before responding. For a once-a-week pattern, that's a minor one-time wait, not a real cost. Open the dashboard a couple of minutes before you start scanning or

sending emails each week to avoid even that.

What's in this guide

Part 0 — What you'll need before starting

Gather these first so you're not stopping halfway through:

- A Gmail address, or better, a Google Workspace account (you mentioned you have one) — this is what will send the 1200 hall ticket emails.
- About 30–45 minutes of uninterrupted time.
- A working internet connection and a web browser (Chrome or Edge).
- This project's files, sitting in this folder on your computer — you don't need to touch the code itself, just follow the steps below.

> Everywhere in this guide you see a grey box like the one below, it's something to copy and paste exactly as shown — don't retype it by hand, small typos in these will stop things from working.

```
Example of a copy-paste box
```

Part 1 — Set up Gmail so emails actually send

Gmail will not let this app log in with your normal password — Google requires a special 16-character "App Password" for anything other than a human typing into gmail.com. This only takes about 5 minutes.

Step 1.1 — Turn on 2-Step Verification (if it isn't on already)

1. Go to myaccount.google.com/security in your browser, signed in as the account you'll send emails from.
2. Find "2-Step Verification" and turn it on if it says "Off". Follow Google's prompts (usually a code sent to your phone).
3. If it already says "On", skip to Step 1.2.

Step 1.2 — Create an App Password

1. Go to myaccount.google.com/apppasswords (you may need to sign in again).
2. Under "App name", type Examflow and click Create.
3. Google will show you a 16-character password like `abcd efgh ijkl mnop`. Copy it down somewhere safe right now — Google only shows it once.
4. Remove the spaces when you use it later, so it becomes one long word, e.g. `abcdefghijklmnop`.

 This app password is not your Gmail password — it's a separate secret code just for this app. Never share it, and don't confuse it with your normal Google sign-in password.

> If your Google account is managed by a school/company (Workspace) and you don't see the "App Passwords" option, ask whoever manages that Google Workspace to enable it for your account, or to enable 2-Step Verification first.

 Write down your Gmail address and this app password now — you'll paste both into Render in Part 5.

Part 2 — Put your project on GitHub

GitHub is a free place to store your project's code online. Render (where we're deploying) reads your code from there.

Step 2.1 — Create a GitHub account

1. Go to github.com and click Sign up. Use any email address and choose a username.
2. Verify your email if asked.

Step 2.2 — Create an empty repository

1. Once signed in, click the + icon (top-right) → New repository.
2. Name it `examflow`.
3. Leave it set to Private (recommended, since this holds real student data) or Public — your choice.
4. Do not check any of the boxes for README/gitignore/license — leave the repository completely empty.
5. Click Create repository. GitHub will show you a page with a web address like `https://github.com/yourname/examflow.git` — copy that address.

→ If Claude is still available in your chat, the easiest option is to paste that GitHub address back into the conversation and ask Claude to push the code for you — it can run the technical commands directly. Otherwise, ask a technical friend to run `git push` for you using the address from Step 2.2.

Part 3 — Create your Render account

1. Go to render.com and click Get Started.
2. Choose Sign up with GitHub — this links your GitHub account from Part 2 automatically, which makes every later step simpler.
3. Approve the connection when GitHub asks.

> Render may ask for a payment card even for free services (common anti-abuse practice) — you should not be charged anything if you follow the Free tiers used in this guide. Please check Render's current pricing page, as this can change over time.

Part 4 — Create the database

This is where all student records, tickets, and attendance get stored. We're using a separate free service called Neon for this instead of Render's own database product, because Neon's free tier never expires — Render's free database gets permanently deleted after 90 days, which would wipe out your student records.

Step 4.1 — Create your Neon account and database

1. Go to neon.tech and click Sign up (signing up with Google is quickest — no credit card required).
2. Create a new project. Name: `examflow`. Choose a region close to your students (e.g. Singapore, if available).
3. Once created, Neon shows you a Connection string that looks like `postgresql://user:password@ep-xxxx.region.aws.neon.tech/neondb?sslmode=require`. Copy the whole thing.

Step 4.2 — Adjust it slightly for this app

This app needs one small change to that connection string before it will work: add `+psycopg2` right after `postgresql`. For example:

```
Neon gives you:
postgresql://user:password@ep-xxxx.neon.tech/neondb?sslmode=require

You paste this into Render instead:
postgresql+psycopg2://user:password@ep-xxxx.neon.tech/neondb?sslmode=require
```

 Keep the `?sslmode=require` part at the end exactly as Neon gave it to you — without it, the connection will fail.

 Neon's free database also "sleeps" after inactivity, like the backend does — but it wakes up in about 1 second on the next request, so in practice you won't notice this one at all.

Part 5 — Deploy the backend (the "engine")

This is the part that actually generates tickets, checks QR codes, and sends emails. Everything else (the two websites) talks to this.

1. On your Render dashboard, click New + Web Service.
2. Choose Build and deploy from a Git repository, then select your `examflow` repository from Part 2 and click Connect.
3. Name: `examflow-backend`
4. Region: same one you picked for the database in Part 4.
5. Root Directory: `backend`
6. Runtime: Render should detect Docker automatically (because of the Dockerfile already in the project) — leave it as Docker.
7. Instance Type: choose Free. Since you send emails in weekly bursts rather than continuously, the free tier's only real tradeoff — a ~30-60 second wake-up if nobody's used it in 15+ minutes — fits your usage fine. See Part 9 for how to avoid even that on exam day.

Step 5.1 — Environment Variables

Scroll down to "Environment Variables" on the same page and add each of the following one at a time (click Add Environment Variable for each row):

Name	Value
`DATABASE_URL`	Paste the adjusted Neon connection string from Part 4, Step 4.2.
`JWT_SECRET_KEY`	eyJhbGciOiJIUzI1NiJ9-EXAMFLOW-2ZQmX9vLpR4tKdWnCsYhBjF7uAeGoTiVs
`JWT_ALGORITHM`	HS256
`JWT_EXPIRE_MINUTES`	720
`SMTP_HOST`	smtp.gmail.com
`SMTP_PORT`	587
`SMTP_USERNAME`	your Gmail/Workspace address from Part 1
`SMTP_APP_PASSWORD`	the 16-character app password from Part 1 (no spaces)
`SMTP_FROM_NAME`	Combine Mentor Official
`TICKET_STORAGE_DIR`	./storage/tickets
`ADMIN_DEFAULT_USERNAME`	admin
`ADMIN_DEFAULT_PASSWORD`	See Part 7 for the password to use here

> The JWT_SECRET_KEY value above is a ready-to-use random secret generated specifically for this guide — you never need to type or remember it, just paste it in once.

Then:

1. Click Create Web Service at the bottom. Render will now build and start your backend — this takes 2–5 minutes the first time.
2. Once it says Live (green), copy the web address shown at the top of the page — it looks like `https://examflow-backend.onrender.com`. You'll need this in Part 6.
3. To check it worked: open that address in a browser with `/health` added to the end (e.g. `https://examflow-backend.onrender.com/health`) — you should see the text `{"status":"ok"}`.

 No extra storage/disk purchase is needed — this app automatically re-creates a student's hall ticket file whenever it's needed, so nothing is lost if Render restarts the backend.

Part 6 — Deploy the two websites (admin + scanner)

These are the two things people actually click around in: the admin dashboard (for you) and the scanner (for the 10 teacher phones). Both are free to host on Render.

Step 6.1 — The Admin Dashboard

1. Click New + → Static Site.
2. Select your `examflow` repository again.
3. Name: `examflow-admin`
4. Root Directory: `frontend-admin`
5. Build Command: `npm install && npm run build`
6. Publish Directory: `dist`
7. Under Environment Variables, add one: Name `VITE_API_URL`, Value = the backend address you copied at the end of Part 5 (e.g. `https://examflow-backend.onrender.com`) — no trailing slash.
8. Click Create Static Site and wait for it to build (1–3 minutes).
9. Copy the resulting address, e.g. `https://examflow-admin.onrender.com` — this is the link you'll open to run your dashboard.

Step 6.2 — The Scanner (for teachers' phones)

Repeat the exact same steps as 6.1, with these differences:

1. Name: `examflow-scanner`
2. Root Directory: `frontend-scanner`
3. Same `VITE_API_URL` environment variable, same backend address as before.

✅ Both of these addresses start with https:// automatically — this is exactly what lets a phone's browser grant camera access for scanning. You do not need to buy a domain name or set up DNS for any of this to work (see the Appendix if you'd like a custom domain anyway).

⚠ VITE_API_URL only gets "baked in" when the site is built. If you ever change your backend's address later, you must update this variable and click "Manual Deploy" → "Clear build cache & deploy" on both the admin and scanner sites again.

Part 7 — Passwords to use

These are chosen to be realistic to remember while still being reasonably hard to guess. Feel free to swap in your own as long as each one mixes letters, a number, and a symbol.

Account	Username	Password
Admin dashboard login	`admin`	`CMO@Exam2026!`
Teacher — Center A, Device 1	`centera.t1`	`CenterA-T1@2026`
Teacher — Center A, Device 2	`centera.t2`	`CenterA-T2@2026`
Teacher — Center A, Device 3	`centera.t3`	`CenterA-T3@2026`
Teacher — Center A, Device 4	`centera.t4`	`CenterA-T4@2026`
Teacher — Center A, Device 5	`centera.t5`	`CenterA-T5@2026`
Teacher — Center B, Device 1	`centerb.t1`	`CenterB-T1@2026`
Teacher — Center B, Device 2	`centerb.t2`	`CenterB-T2@2026`
Teacher — Center B, Device 3	`centerb.t3`	`CenterB-T3@2026`
Teacher — Center B, Device 4	`centerb.t4`	`CenterB-T4@2026`
Teacher — Center B, Device 5	`centerb.t5`	`CenterB-T5@2026`

Where each password goes:

- Admin password paste into `ADMIN_DEFAULT_PASSWORD` in Part 5 before you first create the backend. This becomes your permanent login for the dashboard.
- Teacher passwords you create these yourself, one at a time, from inside the admin dashboard after everything is deployed (see Part 8, Step 8.2) — you type these exact values in there, along with each teacher's real name and which center they belong to.

 Change the admin password to something only you know if you ever suspect it's been seen by someone else — you can do this any time by changing ADMIN_DEFAULT_PASSWORD on Render and restarting the backend, though note this only changes it if the account doesn't already exist; ask Claude for help if you need to reset an existing password.

Part 8 — First-time setup after deployment

Step 8.1 — Log in

1. Open your admin website address from Part 6 (e.g. `https://examflow-admin.onrender.com`).
2. Log in with username `admin` and the password you set in Part 7.

Step 8.2 — Create your 10 teacher accounts

1. Scroll to "Teacher Devices / Accounts" → "Add Teacher".
2. For each of the 10 rows in the Part 7 table: enter the username, the teacher's real full name, the password, and pick their exam center from the dropdown. Click Add.
3. Do this 10 times (5 for each center).

Step 8.3 — Do a small test run before the real thing

1. Prepare a tiny Excel sheet with just 2–3 of your own email addresses instead of real students, with columns: Student Name, Email, Mobile Number.
2. Upload it, pick a center, click Generate Hall Tickets, then Send Emails.
3. Check that the email actually arrives (including checking your Spam folder) with a working PDF attached and a scannable QR code, before uploading your real 1200 students.

Step 8.4 — Upload your real students

1. Prepare your real Excel sheet(s) — one upload per exam center, each with columns Student Name, Email, Mobile Number.
2. Upload each sheet, picking the correct center each time.
3. Click Generate Hall Tickets — with ~1200 students this can take a couple of minutes; it's safe to leave the page and check back, the dashboard will show progress and update automatically.
4. Click Send Emails — this can take longer since it emails one by one; again, safe to leave and come back. Check the Students table afterward for anyone marked "Failed" and click Send Emails again to retry just those (it automatically skips anyone already sent).

→ A Google Workspace account can send about 2000 emails a day, so all 1200 students fit comfortably in a single day.

Part 9 — Exam day checklist

 About 5 minutes before scanning starts, open your admin dashboard address once yourself (or visit your backend address with `/health` added to the end) — this "wakes up" the free backend so the very first student isn't kept waiting.

- Each of the 10 teachers opens the scanner website address from Part 6 on their own phone's browser.
- Each logs in with their own username/password from Part 7.
- The browser will ask for camera permission — teachers must tap Allow.
- Students bring their printed hall ticket (the emailed PDF); the teacher points their phone's camera at the QR code on it.
- Green screen = checked in successfully. Red screen = already scanned today, or wrong exam center — the message on screen explains which.
- You (admin) can watch live attendance numbers update on your dashboard's "Live Attendance" and "Attendance" sections from anywhere, on any device.

Part 10 — If something goes wrong

Problem	What to do
The site feels stuck or times out the first time	This is the free tier waking up (see Part 9's tip box) — wait 30-60 seconds and try again; it will be fast for the rest of that session.
Emails aren't sending / show "Failed"	Double-check SMTP_USERNAME and SMTP_APP_PASSWORD on Render exactly match Part 1 (no spaces in the app password), then restart the backend service (Render dashboard your backend Manual Deploy Deploy latest commit).
Camera won't open on a teacher's phone	Make sure they're using the https:// scanner address from Render (not typed manually), and that they tapped Allow when asked for camera permission. Try closing and reopening the browser tab.
Dashboard shows a blank page or errors	Check that VITE_API_URL on both frontend-admin and frontend-scanner exactly matches your backend's address, then redeploy both (see the warning box at the end of Part 6).
A specific student didn't get their ticket	Search for them in the Students table on the dashboard — check their Ticket and Email Status columns, and click Send Emails again if it shows Failed.
Something else / not covered here	Come back to this Claude Code conversation, describe exactly what you're seeing, and Claude can look at the actual error logs on Render with you.

Part 11 — Self-hosting with Docker Compose (advanced alternative)

Parts 1–10 above cover the recommended path (Render + Neon), which handles HTTPS and servers for you automatically. If you'd rather run everything on your own server/VPS using this project's docker-compose.yml instead, this section covers that path — including the one thing Render gave you for free that a bare server doesn't: HTTPS, which the scanner app requires for camera access.

 Without HTTPS, teachers will see "Could not access camera — camera access is only supported in a secure context (https or localhost)" when they try to scan. This isn't optional for the scanner app.

Step 11.1 — Get the code onto your server

1. Copy this project onto your server (e.g. `git clone` your repository there, or however you normally transfer files to it).
2. Create a `.env` file in the project's root folder — copy `.env.example` and fill in real values (database password, JWT secret, Gmail app password from Part 1, admin password from Part 7).

Step 11.2 — Get a free HTTPS address for your server

Browsers block camera access on any page that isn't served over https:// (localhost is the only exception) — so a bare server IP over plain http will not work for the scanner app. If you don't own a domain name, sslip.io provides a free working hostname for any server, no signup required.

1. Find your server's public IP address (e.g. `203.0.113.10`).
2. Replace the dots with dashes and append `.sslip.io` — e.g. `203.0.113.10` becomes `203-0-113-10.sslip.io`. This automatically resolves straight to your server, with zero setup.

 Already have your own domain? Use it instead (e.g. yourcollege.com) — just point its DNS A record at your server's IP and use that in place of the sslip.io address below.

Step 11.3 — Add the domain variables to your .env

Everything is served off this single domain, split by path: /admin/ for the dashboard, /scanner/ for the scanner PWA, /api/ for the backend. Add these lines to your server's .env file, using your own IP in place of the example:

```
DOMAIN=203-0-113-10.sslip.io
ADMIN_BASE_PATH=/admin/
SCANNER_BASE_PATH=/scanner/
PUBLIC_API_URL=/api
```

Step 11.4 — Open the firewall and start everything

1. Make sure your server's firewall allows inbound traffic on ports 80 and 443 (443 for HTTPS itself, 80 for the automatic certificate check). The individual app ports no longer need to be open — only the reverse proxy (Caddy) is exposed directly now.
2. Run: `docker compose up -d --build`

3 Wait about 10–30 seconds the first time — Caddy automatically fetches a real, trusted HTTPS certificate from Let's Encrypt for the domain, with no manual certificate work.

 You'll know it worked when opening https://<your-ip-with-dashes>.sslip.io/scanner/ on a phone shows a normal padlock icon with no security warning, and the browser asks for camera permission normally instead of refusing outright.

 If you change PUBLIC_API_URL, ADMIN_BASE_PATH, or SCANNER_BASE_PATH later, you must rebuild with `docker compose up -d --build` again — like Render's VITE_API_URL, these get baked into the frontend at build time, not read at runtime.

Appendix — Using your own domain name (optional)

This is entirely optional — everything above works perfectly using the free `.onrender.com` addresses. Only do this if you specifically want an address like `admin.yourcollege.com` instead.

1. Buy a domain name from any registrar (e.g. Namecheap, GoDaddy) — typically $10–15/year.
2. On Render, open your admin (or scanner) static site Settings Custom Domains Add Custom Domain, and enter the address you want.
3. Render will show you one or two DNS records (usually a "CNAME") to add — copy these into your domain registrar's DNS settings page (each registrar's dashboard looks slightly different; their support articles walk through "adding a CNAME record").
4. Wait 10 minutes to a few hours for it to activate — Render will show a green checkmark when it's live.

> DNS is the system that connects a domain name to Render's address — you only need to think about it at all if you choose to do this optional step.
